

UNIVERSITE DE CORSE

2024-2025

Licence SPI 3ème année

Option INFORMATIQUE

UE Qualité Logicielle et Tests

CH4 – Tests



Evelyne VITTORI
vittori@univ-corse.fr

Plan du cours



CH1 – Principes de Qualité



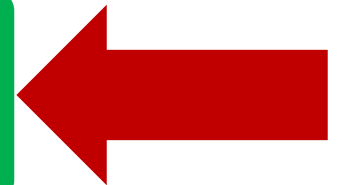
CH2 – Métriques logicielles



CH3 - Bonnes pratiques OO



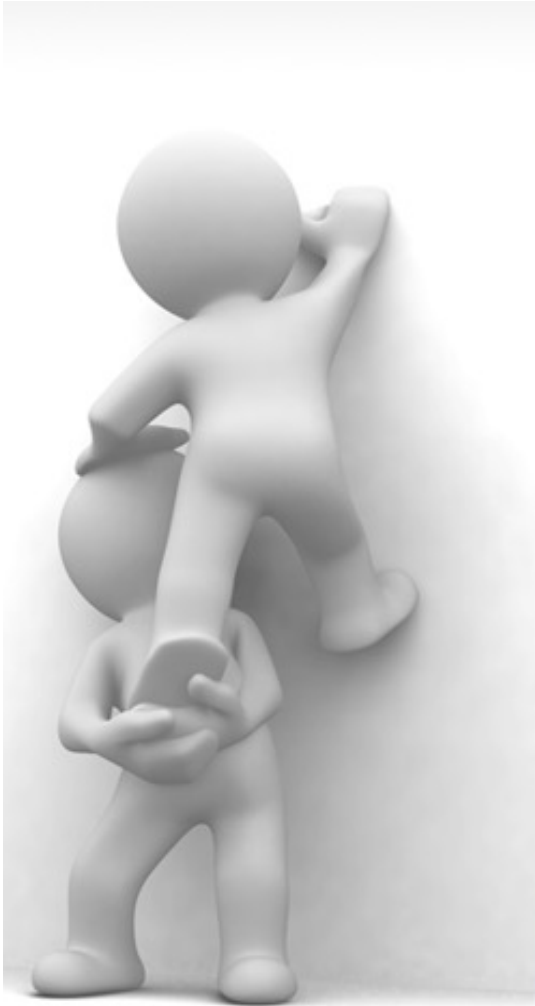
CH4 – Tests





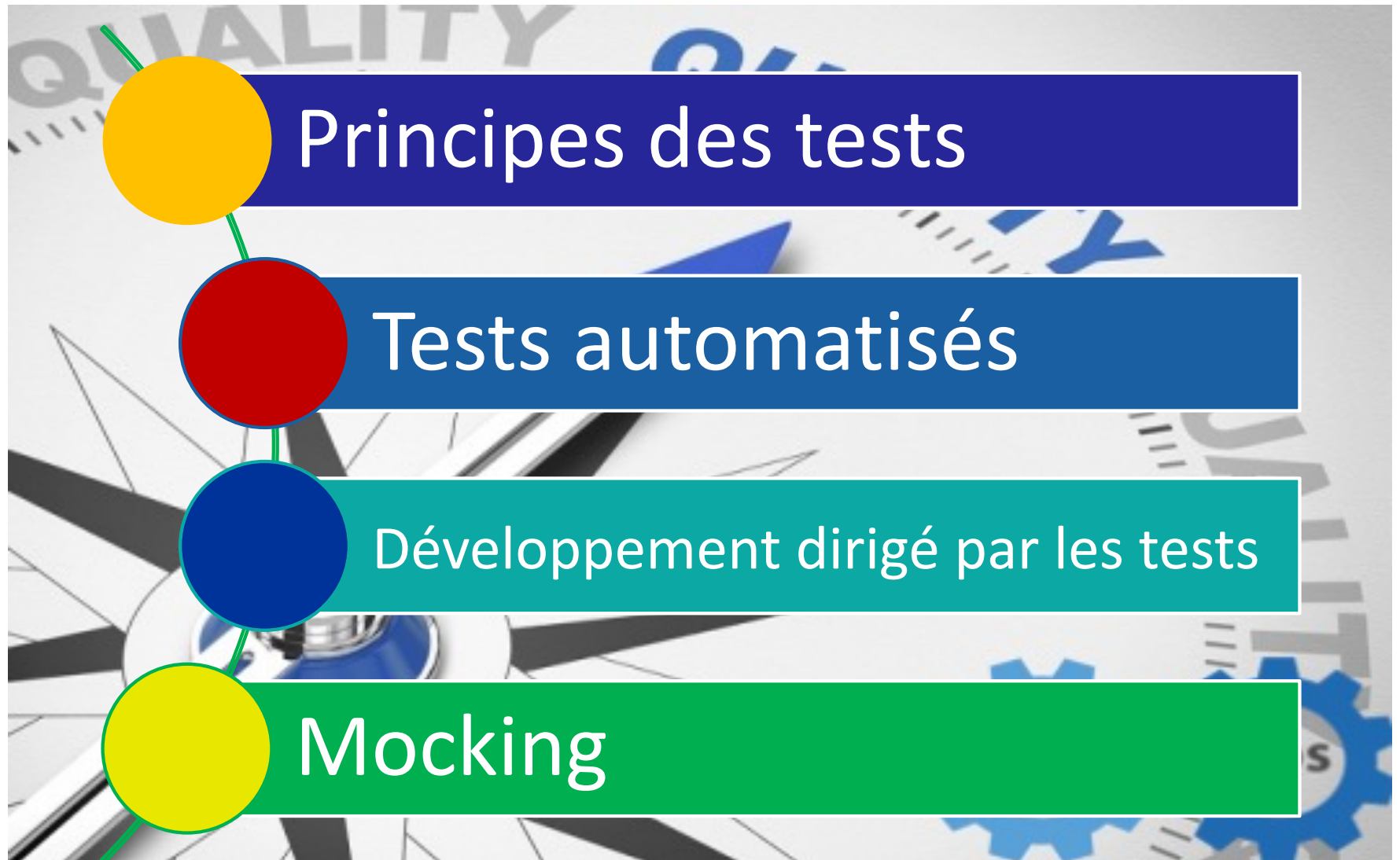
ILLUSTRATED BY SEGUE TECHNOLOGIES

Objectifs de ce chapitre



- Définir la notion de **test**, comprendre son importance et ses différentes dimensions.
- Découvrir le principe des **tests automatisés** et le mettre en œuvre à l'aide de l'outil **JUNIT**.
- S'initier au **développement dirigé** par les tests.
- Découvrir le principe du **mocking** et le mettre en œuvre sur des cas simples.

CH3- Tests





Principes du test logiciel

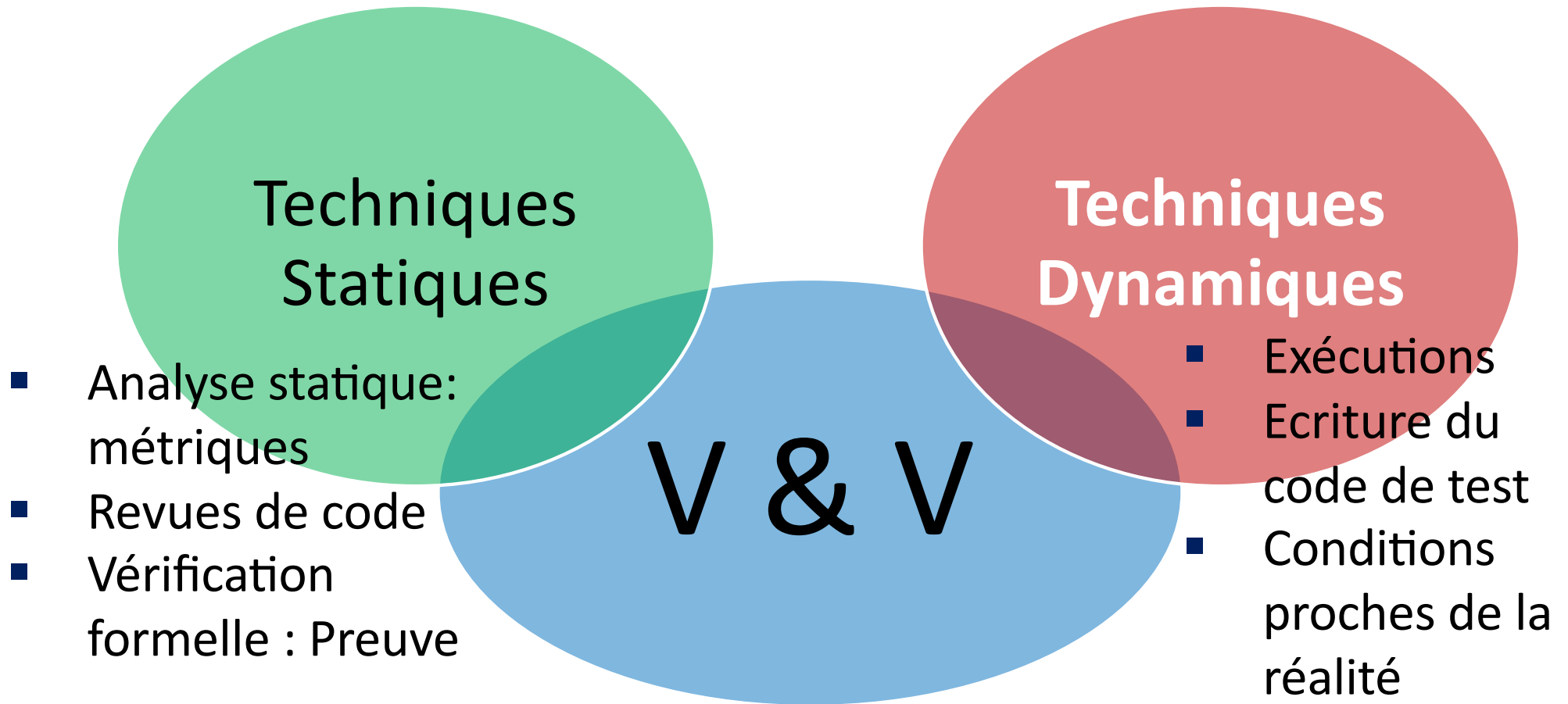
- Qu'est-ce que le test dynamique?
- Qu'est-ce que le test ne permet pas?
- Notions de jeu de test
- Techniques de tests



Notions de V&V

- **Validation** : Est-ce que le logiciel réalise les fonctions attendues par le client?
 - Contrôles en cours de réalisation avec le client
 - Défauts par rapport aux besoins
- **Vérification** : Est-ce que le logiciel fonctionne correctement ?
 - Erreurs par rapport aux spécifications

Approches de la qualité



Actuellement, le test dynamique est la méthode la plus diffusée et représente jusqu'à 60 % de l'effort complet de développement d'un produit logiciel

Qu'est ce que « le test » dynamique?

- Le test est avant tout une recherche d'anomalie!



Program testing can be a very effective way to show the presence of bugs, but is hopelessly inadequate for showing their absence.

Edsger Dijkstra

- Le test permet de détecter des erreurs par l'observation des défaillances qu'elles induisent



Qu'est ce que « le test »?

- Il existe toujours des erreurs mais elles ne sont pas obligatoirement observables
 - Fonction non appelée
 - Notion de **couverture du code** : % du code effectivement testé
- Objectif
 - « 0 anomalie »: irréaliste
 - Plus pragmatique : pas de défaillance sévère ou récurrente

Qu'est ce que le test ne permet pas?

- Trouver la cause des erreurs
- Corriger les erreurs
- Prouver la correction totale d'un programme:
 - Seules les méthodes de preuve formelle peuvent caractériser des preuves incontestables



Mais le test permet quand même d'augmenter notre confiance dans la qualité du logiciel

Tests, essais et débogage

- Un test est un processus documenté qui doit être **reproductible**
- Le test est différent :
 - d'un **essai** ou d'une mise au point
 - du **débogage** qui est une enquête dont le but est d'expliquer un problème



Que veut-on tester?

- **Tests nominaux** (tests de bon fonctionnement)

- Les cas de test correspondent à des données d'entrée valides => **Test-to-pass**

- **Tests de robustesse**

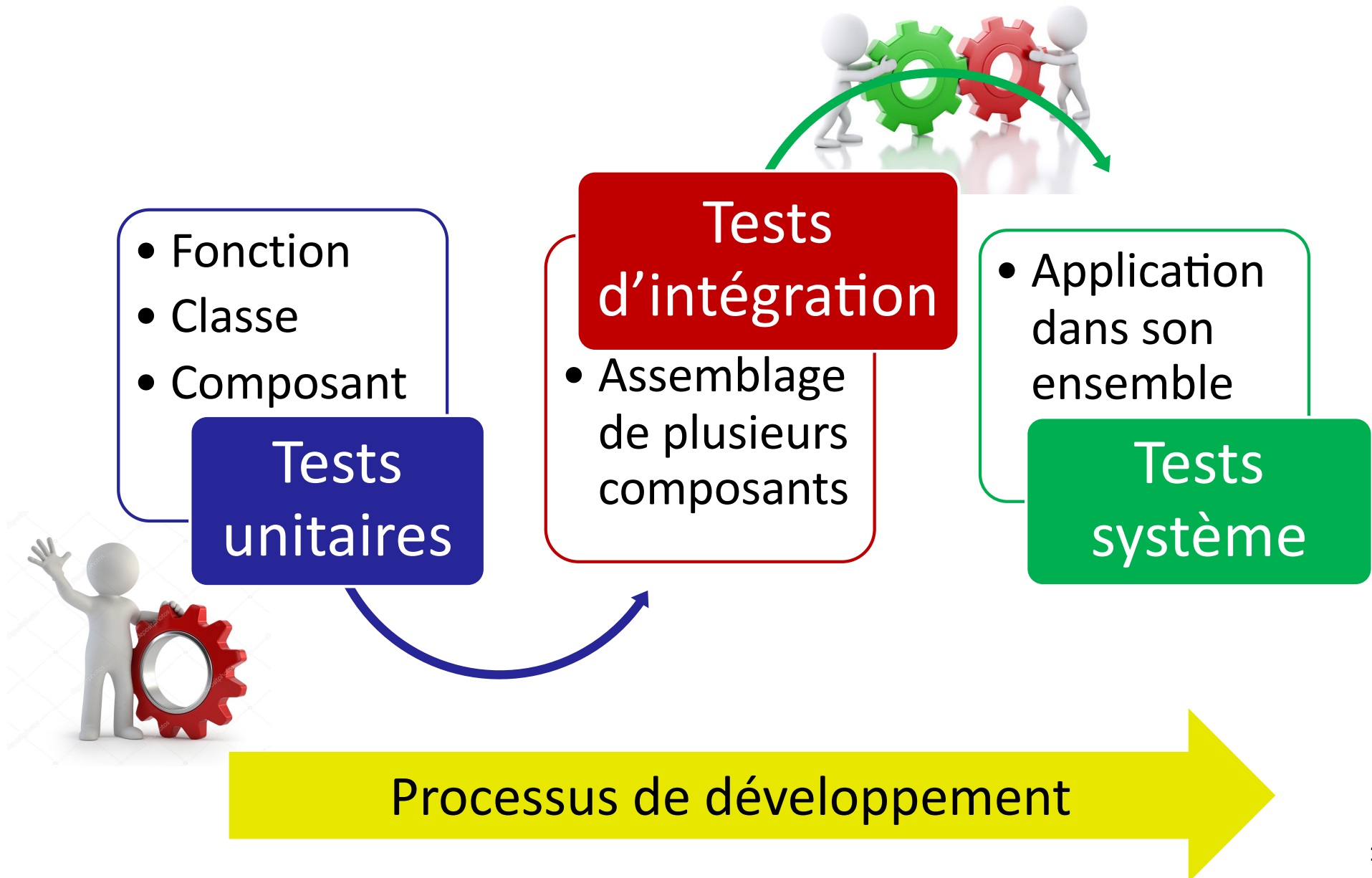
Les tests nominaux sont passés avant les tests de robustesse

- Les cas de test correspondent à des données d'entrée invalides => **Test-to-fail**

- **Test de performance**

- Load testing (test avec montée en charge)
- Stress testing (soumis à des demandes de ressources anormales)

Différents niveaux de test

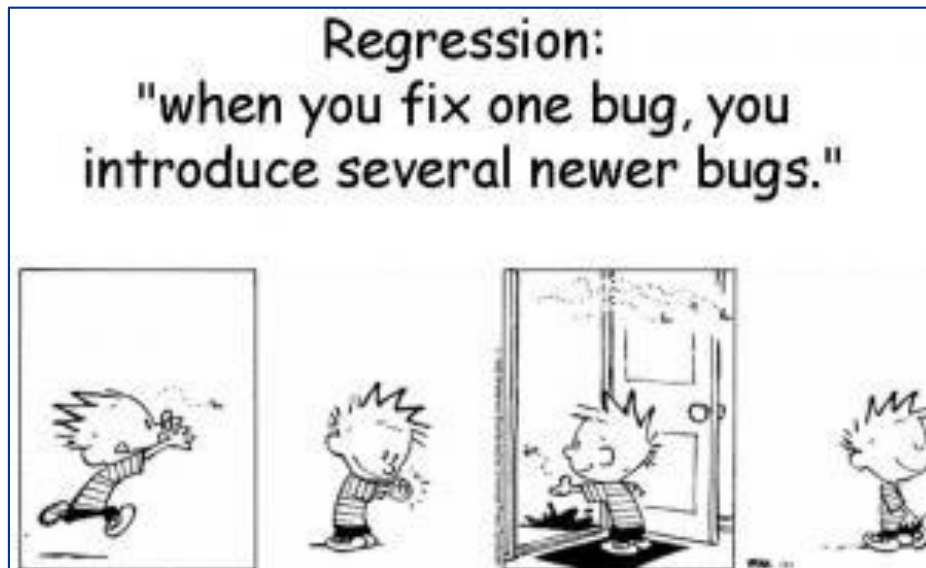


Différents niveaux de tests

Tests de non-regression

- Tests réalisés après une modification des programmes existants
- *On vérifie que l'on n'a pas créé de nouvelles anomalies!!*

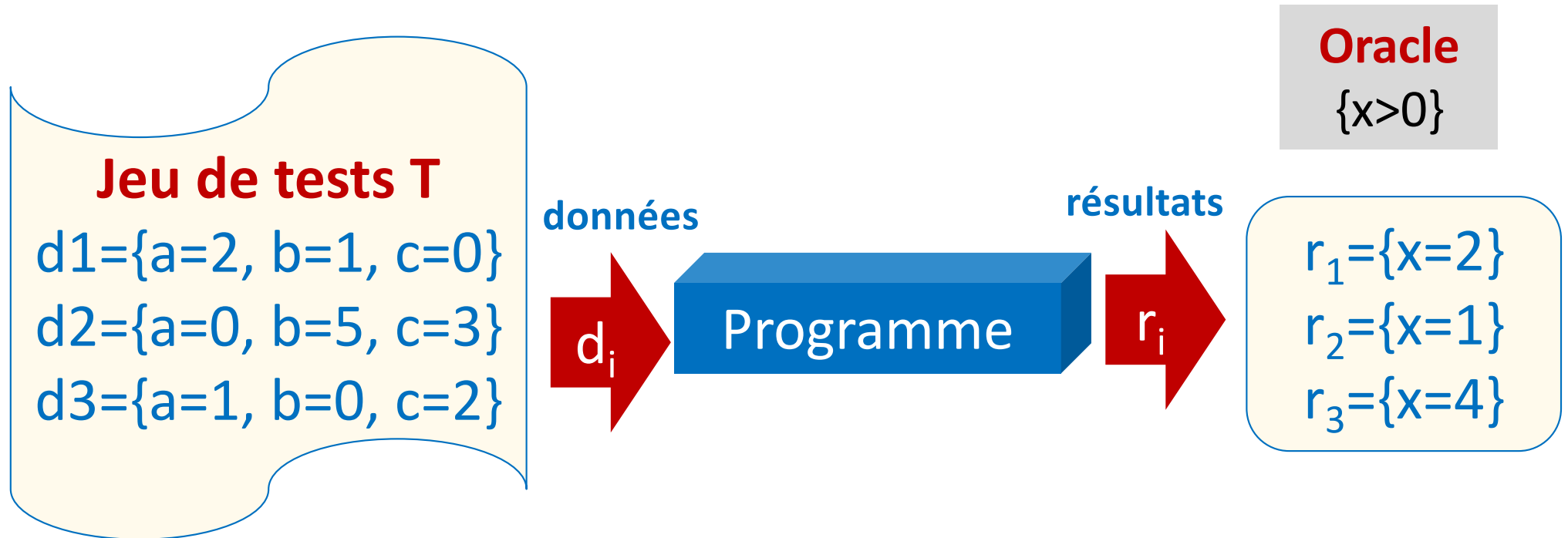
Modifications



Test unitaire

- POO: l'unité est la classe
 - Test unitaire = test d'une classe
- Test du point de vue d'une classe « cliente »
 - les cas de tests appellent les méthodes depuis une classe extérieure
 - on ne peut tester que ce qui est public
- Au moins un cas de test par méthode public
- Plusieurs approches possibles pour identifier les cas de tests
 - Tests boîte noire – Tests Boite blanche

Vocabulaire: Jeux de tests et Oracles



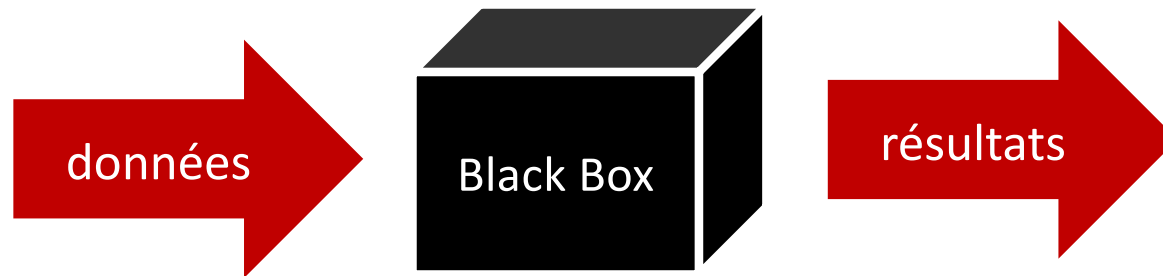
Notion d'Oracle

Condition que les résultats doivent vérifier pour que l'exécution soit correcte

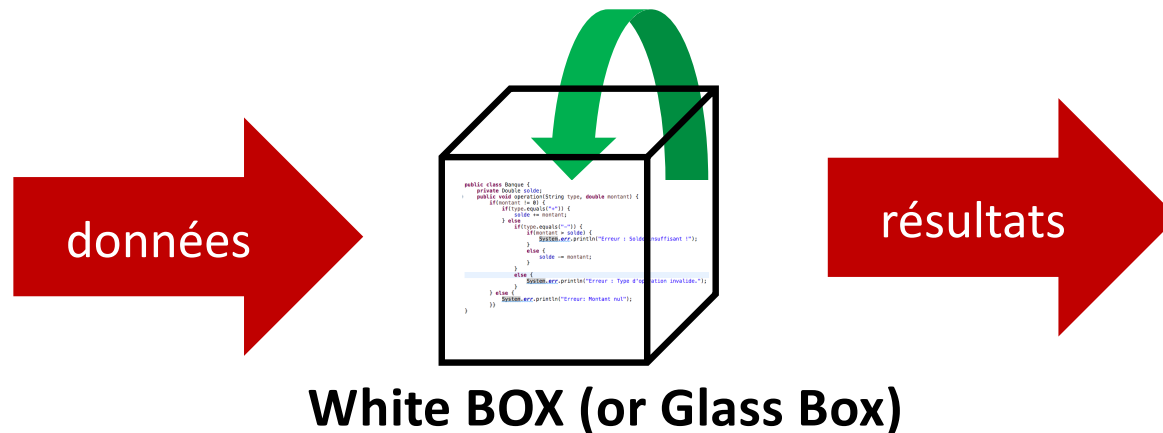
Le test est un succès si les oracles sont vérifiés pour tous les couples (d_i, r_i)

Techniques de tests

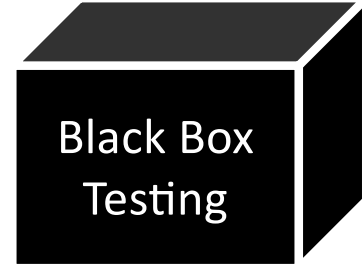
- Tests fonctionnels ou boîte noire



- Test structurels ou boîte blanche



Tests boîte noire



- Les cas de test sont définis de manière fonctionnelle sans examiner le code :
 - perspective externe au programme
 - seule la signature des méthodes et les spécifications sont connues
- Comment déterminer les données de tests (DT)?
 - Technique des partitions d'équivalence
 - Identification des cas de test aux limites

Analyse Partitionnelle

- Données d'entrée

Pour chaque donnée d'entrée:

- Identifier les classes de valeurs valides et invalides
 - Intervalles de valeurs

- Résultats (sorties)

Pour chaque variable de sortie :

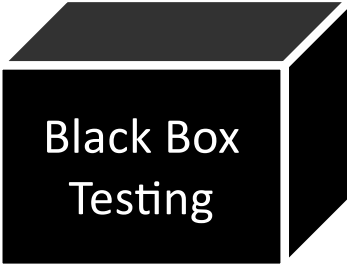
- Identifier les valeurs valides et invalides.

- Identifier les cas de tests pour obtenir une couverture **acceptable** des classes de DT

Une couverture totale est irréaliste

Exemple

Analyse partitionnelle



Black Box
Testing

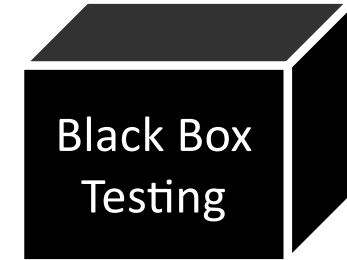
Programme qui détermine si un individu est majeur ou non en fonction de la saisie de son age.

Un age valide est un nombre réel compris entre 1 et 110 inclus .

Classes des ages

- Classe de valeurs valides $V1 : 1 \leq a < 18$
 $V2 : 18 \leq a \leq 110$
- Classe de valeurs Invalides $I1: a < 1$
 $I2 : a > 110$
 $I3 : \text{caractères non numériques}$

	Test1	Test2	Test3	Test4	Test5
Classe Age	V1	V2	I1	I2	I3
Valeur Age	15	20	0,5	120	dix
Résultat	Non majeur	Majeur	Erreur	Erreur	Erreur

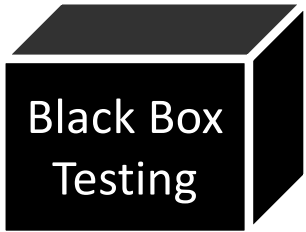


Tests aux limites

- Complémentaire de l'analyse partitionnelle
- Consiste à étudier le comportement du logiciel pour des valeurs aux limites des domaines de définition (aussi appelées *valeurs de bord*):
 - Valeurs très grandes
 - Valeur de boucle nulle, négative, maximale
 - Données non valides
 - Etc...

Exemple : Une variable x dans le domaine $[1,10]$

Valeurs à tester : 0,1,2,9,10,11



Exercice – Calcul d'impôt



1. Identifiez les partitions des données d'entrée (valides et invalides) et de sortie du programme spécifié ci-dessous.
2. En déduire des cas de tests pertinents

*Une application Web calcule le **taux d'imposition** en fonction du **revenu annuel** d'un foyer et de son **nombre d'enfants**.*

Le montant des revenus et le nombre d'enfants (de 0 à 10) sont saisis dans un formulaire et seuls des entiers positifs sont acceptés par le système pour les deux champs.

Les règles de calcul sont les suivantes :

- *Le revenu doit être compris entre 0€ et 100 000€ inclus;*
- *Les revenus de moins de 10 000€ inclus ne sont pas imposables;*
- *Les revenus strictement supérieurs à 10 000€ sont imposés au taux de 20%;*
- *Les foyers avec 2 enfants ou plus ont un taux d'imposition réduit de moitié.*



Exercice (correction)



Partitions des données et résultats

Donnée Revenu R

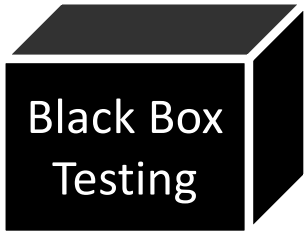
- ✓ Partitions valides
 - RV1 : $0 \leq R \leq 10\,000$
 - RV2 : $10\,000 < R \leq 100\,000$
- ✓ Partitions invalides
 - RI1 : $R < 0$
 - RI2 : $R > 100\,000$
 - RI3 : nombre décimal
 - RI4 : caractère alphabétique

Donnée Nombre d'enfants N

- ✓ Partitions valides
 - NV1 : $0 \leq NB \leq 1$
 - NV2 : $2 \leq NB \leq 10$
- ✓ Partitions invalides
 - NI1 : $NB < 0$
 - NI2 : $NB > 10$
 - NI3 : nombre décimal
 - NI4 : caractère alphabétique

Résultats

- ✓ TauxI = 20%
- ✓ TauxI = 10%
- ✓ TauxI = 0
- ✓ Erreur de saisie (données invalides)



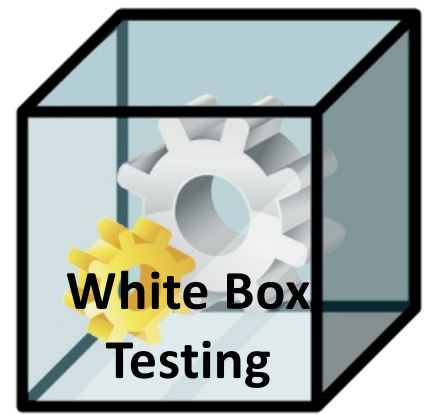
Exercice (correction)



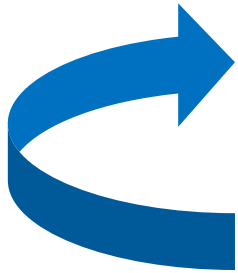
Proposition de Cas de test

	Test1	Test2	Test3	Test4	Test5	Test6	Test7
Revenu	RV1	RV2	RV2	RI1	RI2	RI3	RI4
Valeur R	5000	50000	50000	-1000	150000	5000,5	dix
Nb enfant	NV1	NV1	NV2	NI1	NI2	NI3	NI4
Valeur R	0	1	3	-1	15	1,5	un
Resultat tauxl	0	20%	10%	Erreur	Erreur	Erreur	Erreur

Tests boîte blanche

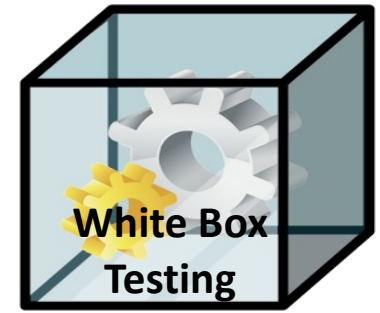


- Tests basés sur la structure du code
- Trois étapes:
 1. Analyse du code
 2. Définition des jeux de tests
 3. Calcul de la couverture du code
 - Identification du code non testé



Utiles pour certains tests unitaires et d'intégration mais non adapté aux tests système

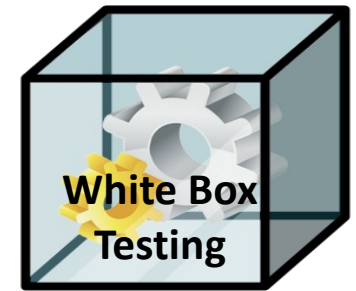
Tests boîte blanche



Comment identifier les cas de tests?

1. Définition du graphe de contrôle cf .CH2.2
2. Calcul de la complexité cyclomatique de Mac Cabe :
 - indication sur le nombre de chemins indépendants
3. Identification des cas de tests (chemins)
 - Couverture des arcs (et des noeuds) : souvent **insuffisant**
 - Couverture des chemins : **irréaliste** pour les cas complexes
 - Il faut trouver une approche intermédiaire:
 - Couverture des arcs
 - Puis étude de l'évolution des variables pour identifier d'autres chemins critiques

Graphe de contrôle et tests



■ Couverture des arcs

- $DT1 = \{x = -2, y = 0\}$: chemin $M1 = \text{a} \text{b} \text{c} \text{d} \text{e} \text{f} \text{g} \text{h}$
- $DT2 = \{x = 1, y = 0\}$: chemin $M2 = \text{a} \text{b} \text{c} \text{e} \text{g} \text{h}$

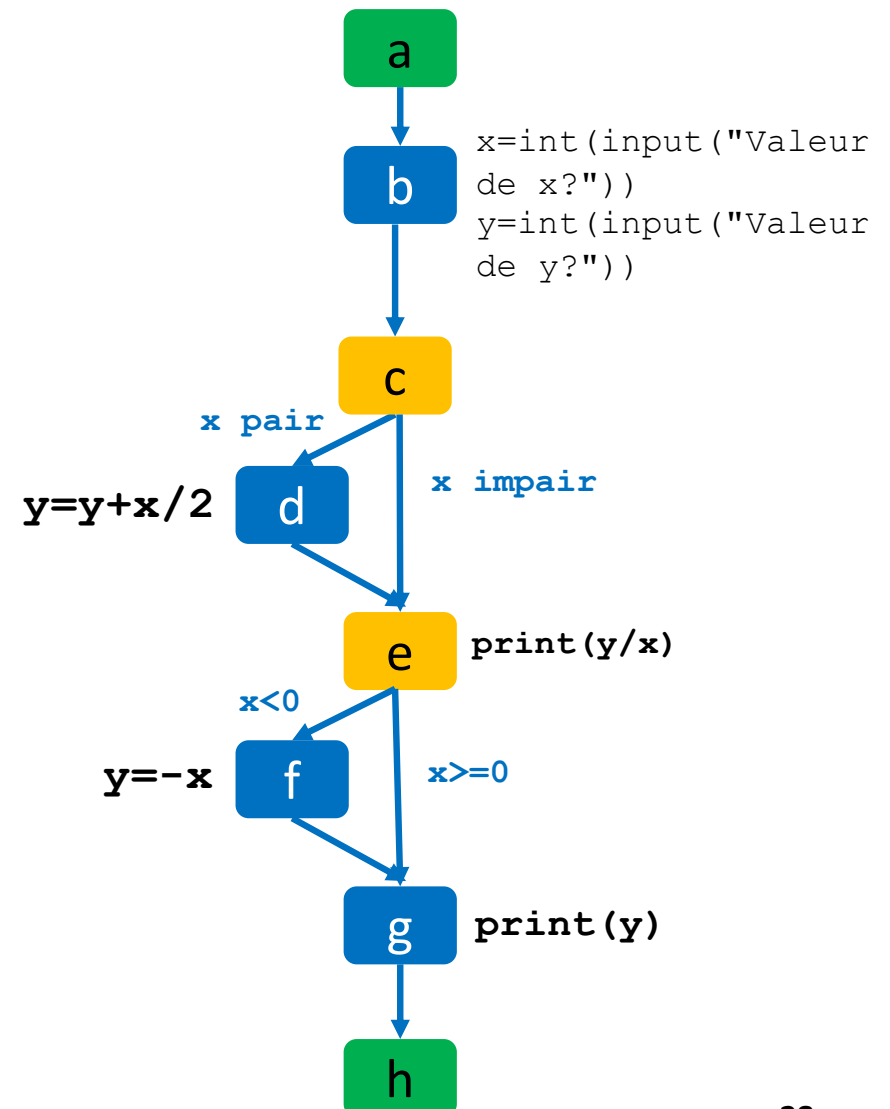
■ Est-ce suffisant?

Non car une erreur au niveau du nœud d ne sera pas détectée par le DT1

- *y est modifié en f donc sa valeur finale ne dépend pas de d*

■ Identification d'un 3ème chemin critique

- $DT3 = \{x = 2, y = 0\}$: chemin $M3 = \text{a} \text{b} \text{c} \text{d} \text{e} \text{g} \text{h}$



Mesure de la couverture de code

- Taux de couverture du code
 - %d'instructions testées
 - %de « branches » testées pour les structures conditionnelles
- Le taux peut être mesuré automatiquement.
- Très utile pour:
 - Contrôler l'avancée des tests.
 - Fixer des objectifs pour les testeurs

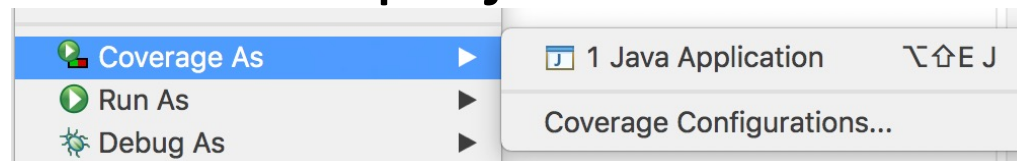
Une couverture élevée ne garantit pas l'absence d'anomalie.
Mais cela permet d'augmenter notre confiance !

Mesure de la couverture de code

Un exemple d'outil dans eclipse

Plugin EclLemma (intégré dans Eclipse)

- Accès par click droit sur le projet

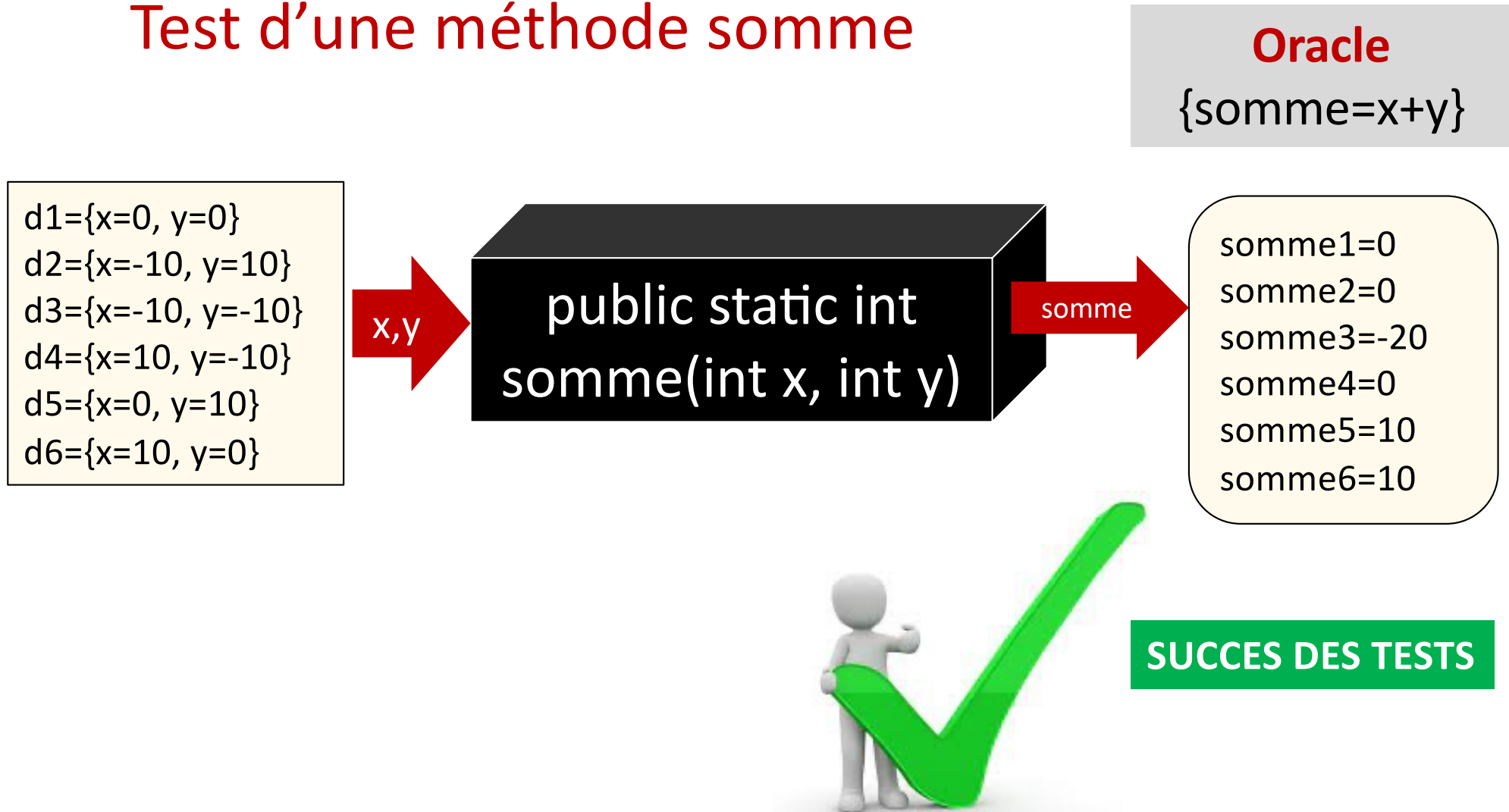


GestionQuizz (28 mars 2021 18:21:15)					
Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ quizzSimple		77,9 %	760	216	976
▼ src		77,9 %	760	216	976
▼ quizz		77,9 %	760	216	976
▶ Clavier.java		27,4 %	32	85	117
▶ GestionQuizz.java		97,7 %	125	3	128
▶ Outils.java		34,8 %	8	15	23
▶ Question.java		100,0 %	114	0	114
▶ QuestionChoixMultiple.java		92,6 %	100	8	108
▶ QuestionChoixUnique.java		70,7 %	58	24	82
▶ QuestionVraiFaux.java		77,1 %	84	24	108
▶ Quizz.java		100,0 %	153	0	153
▶ Solution.java		97,0 %	32	1	33
▶ SolutionQCM.java		100,0 %	7	0	7
▶ SolutionQCU.java		100,0 %	25	0	25
▶ SolutionVF.java		68,8 %	22	10	32
▶ TestQuestion.java		0,0 %	0	45	45

Couverture d'instructions
mais possibilité d'autres
critères

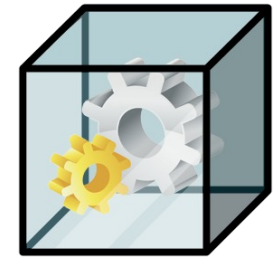
Exemple: test BNoire + test BBlanche

Test d'une méthode somme



Exemple de test boîte blanche

Mesure de la couverture par instructions



Test.java

```
1 package testCouv;
2
3 public class Test {
4
5     public static int somme(int x, int y) {
6         int sum=0;
7         if (x==500 && y==600)
8             sum=x-y;
9         else
10            sum=x+y;
11        return sum;
12    }
13    public static void main(String[] args) {
14        System.out.println(Test.somme(0, 0));
15        System.out.println(Test.somme(0, 10));
16        System.out.println(Test.somme(10, 0));
17        System.out.println(Test.somme(-10, -10));
18        System.out.println(Test.somme(-10, 0));
19        System.out.println(Test.somme(0, -10));
20        System.out.println(Test.somme(10, 10));
21    }
22 }
```

Instruction non testée

Outline

- testCouv
 - Test
 - somme(int, int) : int
 - main(String[]) : void

Probl @ Java Decla Cons

<terminated> Test1 [Java Application] /Library/Java/JavaV

Coverage

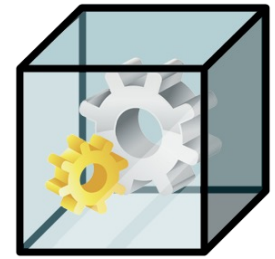
Test1 (28 mars 2021 20:10:42)

Element	Coverage	Covered Instructions	Missed Instructions
testCouverture	81,0 %	47	11
src	81,0 %	47	11
testCouv	81,0 %	47	11
Test.java	81,0 %	47	11
Test	81,0 %	47	11

0
10
10
-20
-10
-10
20

Exemple de test boîte blanche

Mesure de couverture par branches



Comptage des « branches » : 3 non testées

3 of 4 branches missed.

```
1 package testCouv;
2
3 public class Test {
4
5     public static int somme(int x, int y) {
6         int sum=0;
7         if (x==500 && y==600)
8             sum=x-y;
9         else
10            sum=x+y;
11        return sum;
12    }
13    public static void main(String[] args) {
14        System.out.println(Test.somme(0, 0));
15        System.out.println(Test.somme(0, 10));
16        System.out.println(Test.somme(10, 0));
17        System.out.println(Test.somme(-10, -10));
18        System.out.println(Test.somme(-10, 0));
19        System.out.println(Test.somme(0, -10));
20        System.out.println(Test.somme(10, 10));
21    }
22 }
```

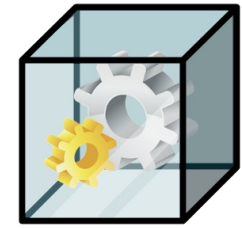
Coverage

Test1 (1 avr. 2021 15:17:36)

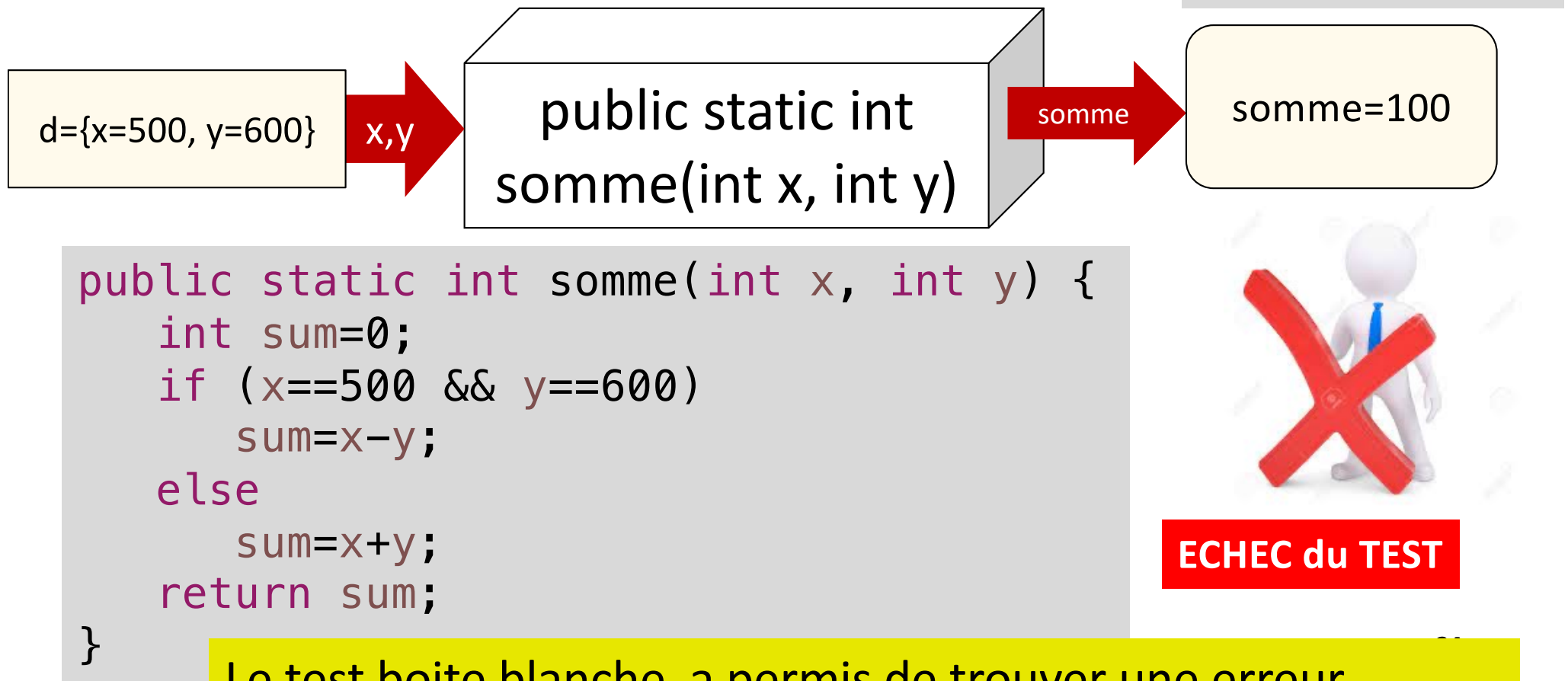
Element	Coverage	Covered Branches	Missed Branches	Total Branch
testCouverture	25,0 %	1	3	
src	25,0 %	1	3	
testCouv	25,0 %	1	3	
Test.java	25,0 %	1	3	
Test	25,0 %	1	3	

- Show Projects
- Show Package Roots
- Show Packages
- Show Types
- Instruction Counters
- ✓ Branch Counters
- Line Counters
- Method Counters
- Type Counters
- Complexity

Exemple de test Boite Blanche



Test de la méthode somme



Le test boite blanche a permis de trouver une erreur difficile à trouver avec les tests boite noire

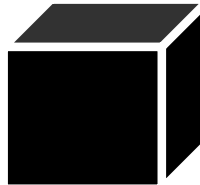
Les tests
Boite noire
et
Boite blanche
sont
complémentaires



Difficulté du test

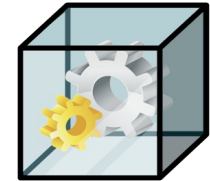
Le test exhaustif est **irréaliste** dans la plupart des cas.

■ Test fonctionnel



L'ensemble des données d'entrée est en général infini ou de très grande taille

■ Test structurel



Le parcours du graphe de flot de contrôle conduit à une forte explosion combinatoire du nombre de chemins

- ✓ Le test est une méthode de vérification partielle de logiciels
- ✓ La qualité du test dépend de la pertinence du choix des données de test

Exercice : Etude de la couverture de code



On considère une méthode statique *calculsX*

■ ***public static int calculsX(int x)***

//Si x est positif ou nul, la méthode renvoie x+1 et si x est négatif, elle renvoie sa valeur absolue +2.

Les tests en boîte noire ont validé la méthode à partir des valeurs DT1, DT2 et DT3.

Cas de test (valeurs de x)

DT1 = -5 DT2 = 0 DT3 = 5

1. Sans regarder le code mais en examinant uniquement la signature de la méthode et sa spécification, que pensez-vous des cas de tests identifiés?

Exercice : Etude de la couverture de code



2. Récupérez la classe Calculs sur l'ENT

- Complétez-la en définissant le programme de test correspondant.
- Utilisez un outil pour évaluer la couverture de code de votre programme de test.
- Quels résultats obtenez-vous (branches et instructions)?

3. Dessinez le graphe de contrôle de la méthode, identifiez les cas de tests à ajouter.

Remarque : En dehors de ce problème, identifiez deux maladroresses dans l'écriture du code.

```
public static int calculsX(int x) {  
    if (x < 0)  
        x = -x;  
    else  
        x -= 1;  
    if (x == 2)  
        x = 1;  
    else  
        x += 2;  
    return x;  
}
```



Test automatisés

- Principes
- Exemple de framework de test: JUnit



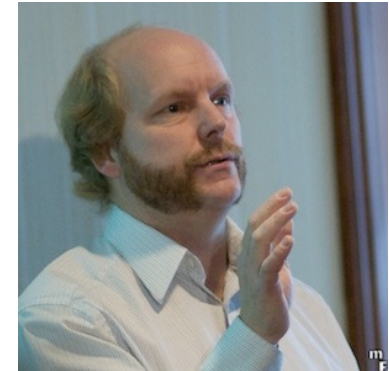
Qu'est ce qu'un framework de test?

- Un ensemble d'outils pour :
 - Structurer les tests
 - Simplifier l'écriture des tests
 - Faciliter la maintenance des tests
 - Exécuter les tests de manière automatique
- Une première étape vers le Test Driven Development.

JUNIT



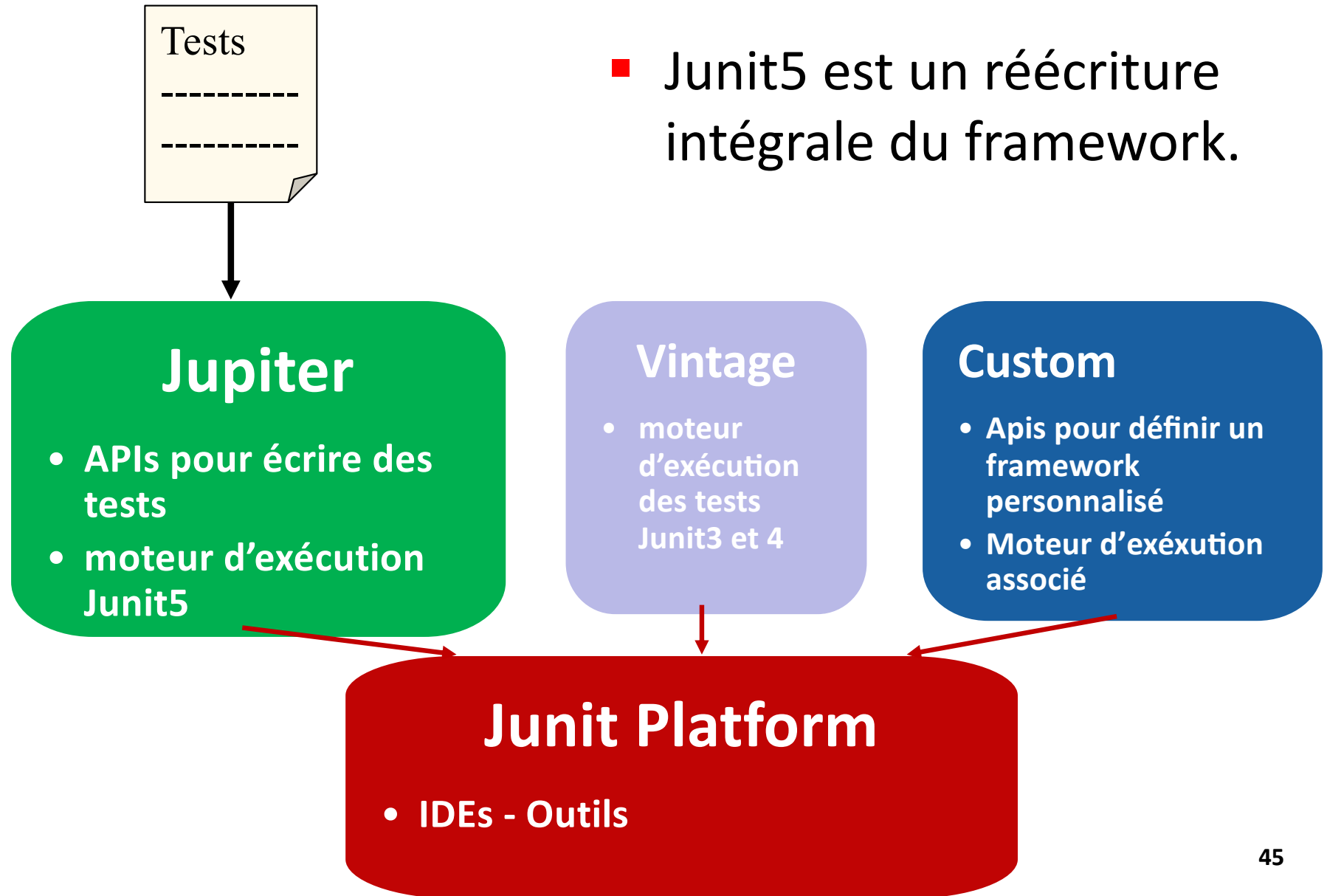
Erich. Gamma



Kent. Beck

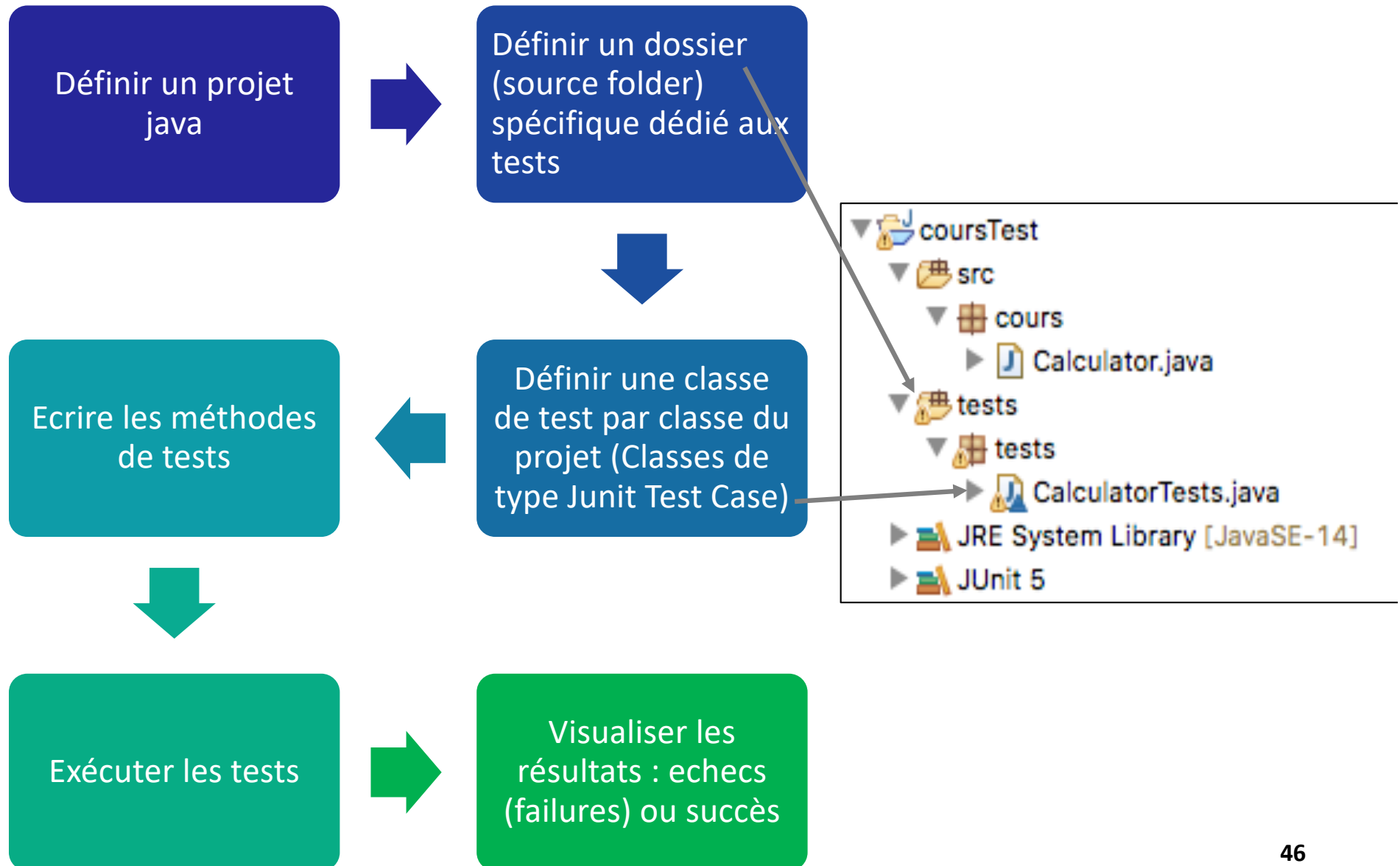
- Framework de tests Open source écrit en Java www.junit.org
- Junit est Membre de la famille XUnit (CUnit pour C, PHPUnit pour PHP...) :
 - Intégré dans Eclipse
 - Pour l'installer dans VSCode (extension javaTestRunner)
 - <https://code.visualstudio.com/docs/java/java-testing>)
- Origine : mouvement Agile
- Basé sur la notion **d'assertion**
- Version actuelle : JUnit5 (nécessite java>=8)

Architecture



- Junit5 est un réécriture intégrale du framework.

Les étapes du test en Junit5



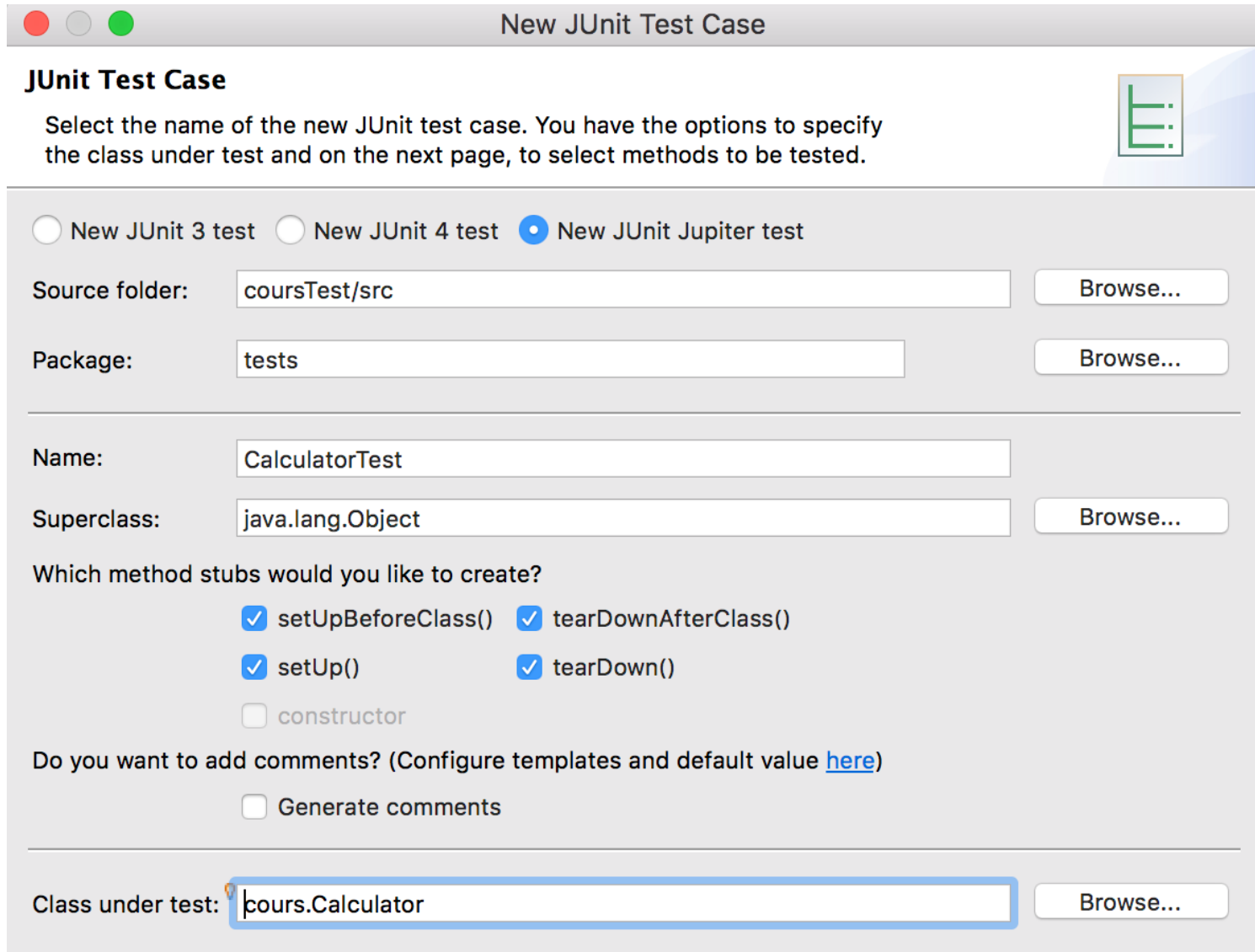
Classe Junit TestCase

- Les tests d'une classe A sont regroupés dans une classe de type *Junit TestCase*.



- La classe ATest de type *Junit Testcase* contient des méthodes de test des méthodes de la classe A.
 - Méthodes avec des annotations spécifiques (ex: @test)

Création d'une classe de test



New JUnit Test Case

JUnit Test Case

Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.

☐ New JUnit 3 test ☐ New JUnit 4 test ☒ New JUnit Jupiter test

Source folder:

Package:

Name:

Superclass:

Which method stubs would you like to create?

☒ setUpBeforeClass() ☒ tearDownAfterClass()
☒ setUp() ☒ tearDown()
☐ constructor

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

Class under test:

Un premier exemple simple

Définition d'une classe de test

Calculator : Classe Java

```
package cours;

public class Calculator {

    public int add(int a, int b){
        return a + b;
    }
}
```

CalculatorTest : Classe de test

```
package tests;

class CalculatorTests {

    @Test
    public void testAdd() {
        Calculator calculator = new Calculator();
        assertEquals(2, calculator.add(1, 1));
    }
}
```

annotation

Assertion = ORACLE

Un premier exemple simple

Exécution d'une classe de test

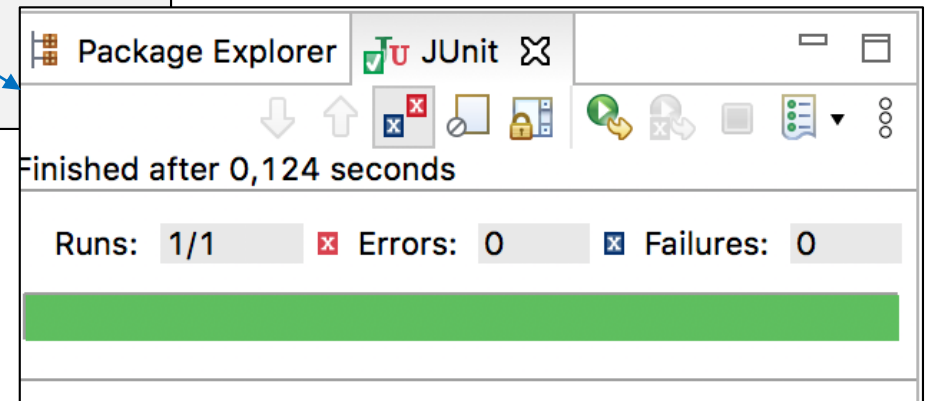
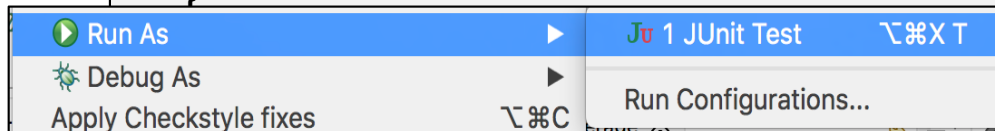
CalculatorTests : Classe de test

```
package tests;

class CalculatorTests {

    @Test
    public void testAdd() {
        Calculator calculator = new Calculator();
        assertEquals(2, calculator.add(1, 1));
    }
}
```

Pour lancer l'exécution des tests :
Clic droit sur la classe de test puis
Run As -> JUnit Test



Méthode de tests

- Obligatoirement **void** et non privées
- Identifiées par une **annotation**
- Exécutées **automatiquement** lors de l'exécution de la classe de test
- Pour vérifier les résultats attendus, il faut écrire des oracles :
 - Utiliser les méthodes **static assertXXX()** fournies dans la classe Assertions du package `org.junit.jupiter.api`

Structure d'une méthode de test

L'ensemble de ces
objets est appelé :
FIXTURE du test

```
@Test  
public void testAdd() {
```

1 - Code de mise en place du contexte
Définition des objets de tests

```
Calculator calculator = new Calculator();
```

2 - Expérimentation sur les objets du
contexte

```
int result= calculator.add(1, 1);
```

3 - Vérification du résultat, Oracle

```
assertEquals(result,2);
```

```
}
```

Assertions

Les assertions sont des méthodes statiques de la classe **Assertions** du package org.junit.jupiter.api

- assertEquals(expected,calculated,message)
- assertEquals(expected,calculated)
- assertNotEquals(...)
- assertNotNull(exp) assertNull(exp)
- assertTrue(exp) assertFalse(exp)
- assertIterableEquals(expected, list)
- assertArrayEquals(t1, t2)

Pour en savoir plus, consulter la javadoc de la classe Assertions:

<https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

Types de méthode de tests

- Deux types de méthodes:
 - **Méthodes de tests simples** (d'instance)
 - @test
 - Mais aussi : @RepeatedTest, @ParameterizedTest.
 - **Méthodes « Lifecycle »**
 - Méthodes statiques: @BeforeAll et @AfterAll
 - Méthodes d'instance: @BeforeEach et @AfterEach

Structure d'une classe de test

```
class CalculatorTest {  
    //Attributs de la classe: Objets de la classe à tester  
    Calculator c;
```

```
@BeforeAll
```

Méthode exécutée avant les tests

```
static void initAll() throws Exception {  
}
```

```
@BeforeEach
```

Méthode exécutée avant chaque test

```
void init() throws Exception {  
}
```

```
@AfterEach
```

Méthode exécutée après chaque test

```
void tearDown() throws Exception {  
}
```

```
@AfterAll
```

Méthode exécutée après la fin des tests

```
static void tearDownAll() throws Exception {  
}
```

```
@Test
```

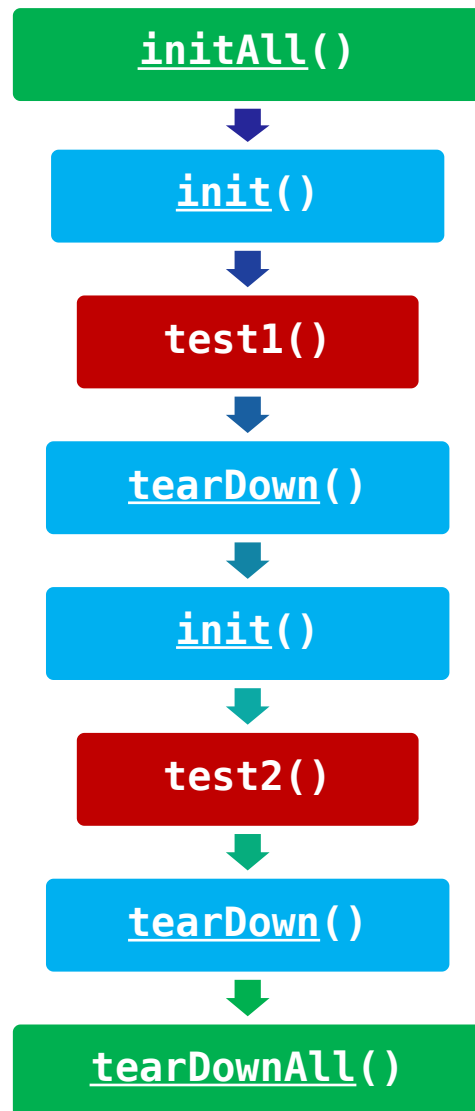
```
void test() {  
    fail("Not yet implemented");  
}
```

```
}
```

Initialisation du
contexte

Méthode de test

Ordre d'exécution des méthodes



```
class CalculatorTest {
    @BeforeAll
    static void initAll() throws Exception {
    }
    @AfterAll
    static void tearDownAll() throws Exception {
    }
    @BeforeEach
    void init() throws Exception {
    }
    @AfterEach
    void tearDown() throws Exception {
    }
    @Test
    void test1() {
        fail("Not yet implemented");
    }
    @Test
    void test2() {
        fail("Not yet implemented");
    }
}
```

Un autre exemple

```
public class Money {
    private int montant;
    private String devise;
    public Money(int montant, String devise) {
        this.montant= montant;
        this.devise= devise;
    }
    public int getMontant() {
        return montant;
    }
    public String getDevise() {
        return devise;
    }
    public Money ajouter(Money m) {
        return new Money(montant+m.getMontant(),
            this.getDevise());
    }
    public boolean equals(Object anObject) {
        if (anObject instanceof Money) {
            Money aMoney= (Money)anObject;
            return
                aMoney.getDevise().equals(getDevise())
                && getMontant() == aMoney.getMontant();
        }
        return false;
    }
    @Override
    public String toString() {
        return "Money [montant=" + montant + ", devise="
    }
}
```

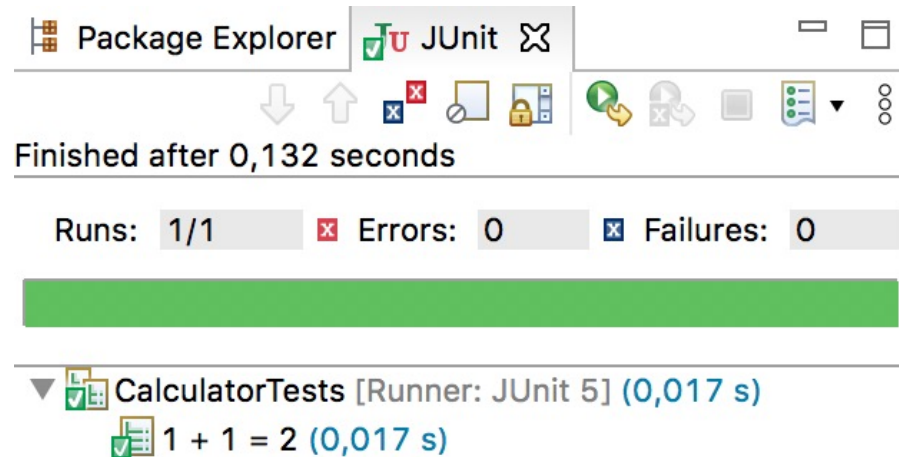
```
public class MoneyTest {
    private Money f12Euro;
    private Money f14Euro;

    @BeforeEach
    public void setUp() {
        f12Euro= new Money(12, "Euro");
        f14Euro= new Money(14, "Euro");
    }

    @Test
    public void testEquals() {
        assertTrue(!f12Euro.equals(null));
        assertEquals(f12Euro, f12Euro);
        assertEquals(f12Euro, new Money(12,
            "Euro"));
        assertTrue(!f12Euro.equals(f14Euro));
    }

    @Test
    public void testSimpleAjout() {
        Money expected= new Money(26, "Euro");
        Money result= f12Euro.ajouter(f14Euro);
        assertTrue(expected.equals(result));
    }
}
```

Nommage d'un test



Nom du test
Affiché lors de l'exécution

Classe de test

```
package tests;

class CalculatorTests {

    @Test
    @DisplayName("1 + 1 = 2")
    void testAdd() {
        Calculator calculator = new
            Calculator();
        assertEquals(2, calculator.add(1, 1),
            "1 + 1 doit être égal à 2");
    }
}
```

Exercice : Test calculatrice



1. Récupérez sur l'ENT le code de la classe Calculator.
2. Créez un projet Java contenant la classe Calculator
3. Créez un sourceFolder spécifique pour les classes de test.
4. Ajoutez-y une classe CalculatorTest de type Junit TestCase et définissez deux méthodes de test de la méthode prod testant les DT (10,2) et (10,0) en vous inspirant de la méthode de test testAdd.

```
class CalculatorTests {  
    private final Calculator calculator = new Calculator();  
    @DisplayName("Test add")  
    @Test  
    void testAdd() {  
        assertEquals(2, calculator.add(1, 1));  
    } . ....  
}
```

5. Exécutez vos tests et examinez la couverture de code de vos tests en utilisant l'outil Coverage d'Eclipse.
6. Complétez les tests pour obtenir une couverture de 100% de la méthode prod
7. Complétez les tests par une méthode de test de la méthode div testant les DT (10,0). Que constatez vous?

Test d'un déclenchement d'exception



- Méthode **assertThrows**
 - `assertThrows(classe d'exception, executable)`
 - Classe d'exception: classe de l'exception qui doit être levée
 - Executable: `() -> object.methodeLevantException()`
 - Le test est réussi si l'exception précisée est levée
 - Dans le cas contraire (aucune exception n'est levée ou une exception d'un autre type est levée) le test échoue.

Test d'un déclenchement d'exception



```
public class Money {
    private int montant;
    private String devise;
    public Money ajouter(Money m)
        throws DeviseException{
        if (!m.getDevise().equals(this.getDevise())) {
            throw new DeviseException(this, m);
        }
        else {
            return new Money(montant+m.getMontant(),
                this.getDevise());
        }
    }
}
```

```
public class DeviseException
    extends Exception {
    private Money m1, m2;
    public DeviseException(Money m1,
        Money m2) {
        this.m1=m1;
        this.m2=m2;
    }
    public String toString() {
        return "Devises incompatibles : " +
            m1.getDevise() + " != " +
            m2.getDevise();
    }
}
```

```
public class MoneyTest {
    private Money f12Euro;
    private Money f14Euro;
    private Money f14Dollar;
    @BeforeEach
    public void setUp() {
        f12Euro= new Money(12, "Euro");
        f14Euro= new Money(14, "Euro");
        f14Dollar= new Money(14, "Dollar");
    }
    @Test
    public void testSimpleAjout() throws
        DeviseException {
        Money expected= new Money(26, "Euro");
        Money result= f12Euro.ajouter(f14Euro);
        assertTrue(expected.equals(result));
    }
    @Test
    public void leveExceptionPourAddition() {
        assertThrows(DeviseException.class,
            () -> f12Euro.ajouter(f14Dollar) );
    }
}
```


Exercice : Test calculatrice(suite)



1. Définissez une nouvelle classe d'Exception **DivZeroException**.
2. Modifiez votre classe Calculator afin qu'une exception de type **DivZeroException** soit levée par la méthode div si le paramètre (y) correspondant au diviseur est égal à 0.
3. Ajoutez une méthode de test de la levée de cette exception dans votre classe de test.
4. Exécutez votre code de test.

Exercice : Test calculatrice(suite)

(Correction)



1. Définissez une nouvelle classe d'Exception DivZeroException.
2. Modifiez votre classe Calculator afin qu'une exception de type DivZeroException soit levée par la méthode div si le paramètre (y) correspondant au diviseur est égal à 0.

```
public int div(int x, int y) throws DivZeroException {  
    if (y==0) {  
        throw new DivZeroException();  
    }  
    return x / y;  
}
```

3. Ajoutez une méthode de test de la levée de cette exception dans votre classe de test.

```
@Test  
void testDivZeroException() {  
    assertThrows(DivZeroException.class, () -> calculator.div(10, 0));  
}
```

Tests répétés



- L'annotation `@RepeatedTest(n)` permet de spécifier la répétition d'une méthode de test un certain nombre de fois

```
class CalculatorTests {  
    @DisplayName("test addition répété");  
    @RepeatedTest(3)  
    public void testAdd() {  
        Calculator calculator = new Calculator();  
        assertEquals(2, calculator.add(1, 1));  
    }  
}
```

Le test est répété 3 fois

Tests paramétrés



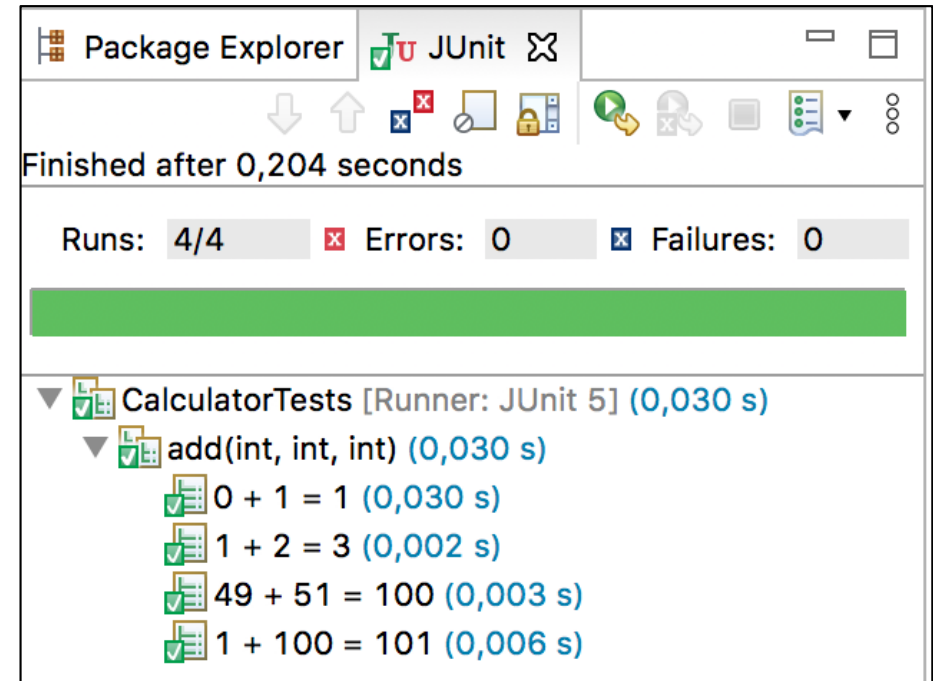
- L'annotation **@ParameterizedTest** permet de spécifier la répétition d'une méthode de test paramétrée en lui associant une liste de paramètres.
- Les valeurs successives des paramètres (source des données) sont définies dans des annotations **@..Source**.

Annotation	contenu
@ValueSource	Tableau de chaîne de caractères ou de types primitifs
@EnumSource	Enumération
@MethodSource	Résultat d'une méthode
@CsvSource	chaînes de caractères dans laquelle chaque argument est séparé par une virgule
@CsvSourceFile	un ou plusieurs fichiers CSV

Exemple de test paramétré



```
class CalculatorTests {
    @ParameterizedTest(name = "{0} + {1}
        = {2}")
    @CsvSource({
        "0,    1,    1",
        "1,    2,    3",
        "49,   51, 100",
        "1,   100, 101"
    })
    void add(int first, int second, int
        expectedResult) {
        Calculator calculator = new
            Calculator();
        assertEquals(expectedResult,
            calculator.add(first, second),
            () -> first + " + " + second + "
                should equal " + expectedResult);
    }
}
```



Exemple de test paramétré



```
class CalculatorTests {  
    @ParameterizedTest(name =  
        "{1} = {2}")  
    @CsvSource({  
        "0, 1, 1",  
        "1, 2, 3",  
        "49, 51, 100",  
        "1, 100, 102"  
    })  
    void add(int first, int second,  
            int  
            expectedResult) {  
        Calculator calculator = new  
            Calculator();  
        assertEquals(expectedResult,  
            calculator.add(first, second,  
                () -> first + " + " + second  
            + " should equal " +  
            expectedResult);  
    }  
}
```

Package Explorer JUnit

Finished after 0,208 seconds

Runs: 4/4 Errors: 0 Failures: 1

CalculatorTests [Runner: JUnit 5] (0,029 s)

- add(int, int, int) (0,029 s)
 - 0 + 1 = 1 (0,029 s)
 - 1 + 2 = 3 (0,003 s)
 - 49 + 51 = 100 (0,002 s)
 - 1 + 100 = 102 (0,007 s)

Failure Trace

org.opentest4j.AssertionFailedError: 1 + 100 should equal 102 ==> expected: <102> but was: <101>

at tests.CalculatorTests.add(CalculatorTests.java:38)

at java.base/java.util.stream.ForEachOps\$ForEachOp\$OfRef.accept(ForEachOps.java:183)

at java.base/java.util.stream.ReferencePipeline\$3\$1.accept(ReferencePipeline.java:195)

Qualités des tests automatisés

Pour être utiles, les tests automatisés doivent répondre à certains critères :

- Être écrits aussi simplement que possible afin
 - d'être compréhensibles et aisément modifiables
- Être auto-vérifiables et renouvelables :
 - sans intervention humaine
 - toujours produire le même résultat
- Couvrir tous les besoins du système
- Ne pas être redondants
- Être très spécifiques : chaque test traite un point précis
- Être Indépendants les uns des autres: aucun ordre d'exécution

Ré-exécutés en cas de modification pour assurer la non-régression

Les mocks pourront aider à tester les classes de manière isolée (cf. TDD)

Exercice : Test calculatrice(suite2)



1. En utilisant la fonctionnalité des tests paramétrés introduite par JUnit5, définissez une méthode de test paramétrée de la méthode prod de la classe Calculator.
2. Exécutez votre code de test.



Développement Dirigé par les tests

- Principe
- Mise en oeuvre

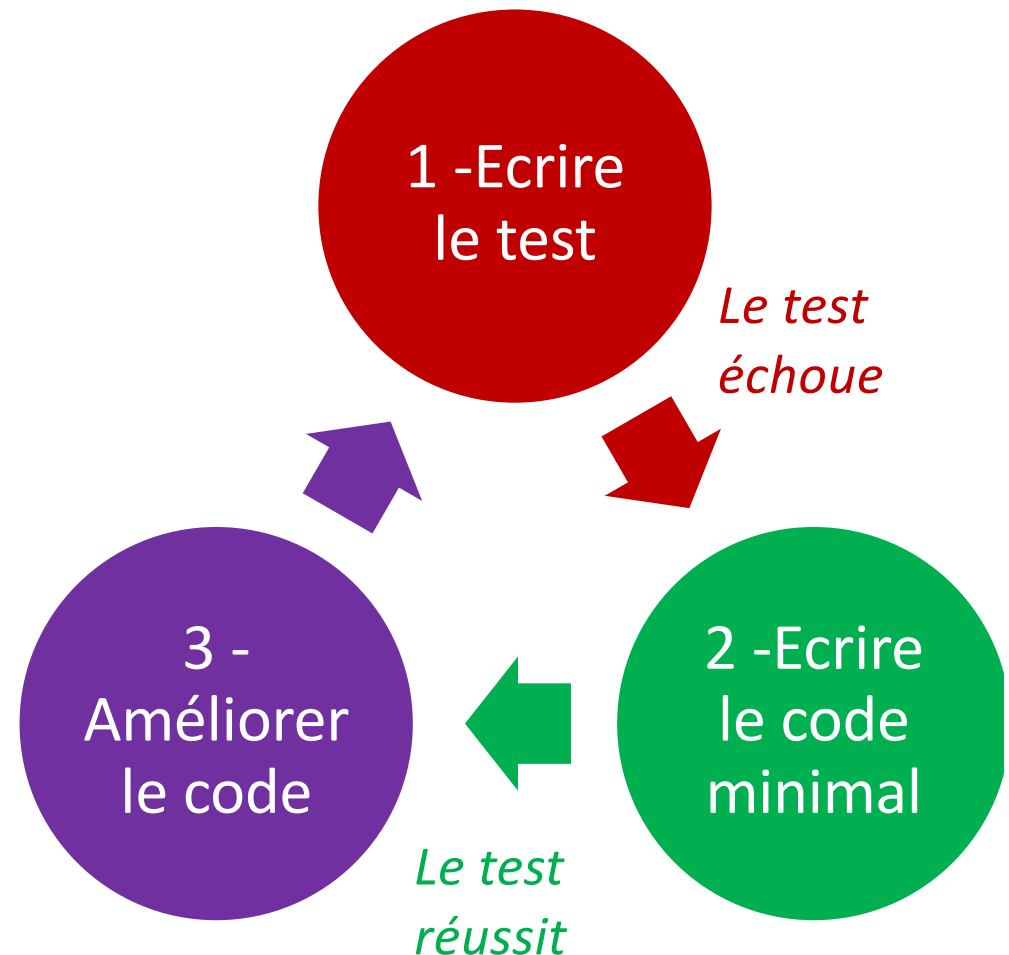
TDD
ALL CODE IS GUILTY
UNTIL PROVEN INNOCENT



UNIVERSITÀ DI CORSICA
PASQUALE PAOLI

Principe du TDD

- Le TDD consiste à écrire les tests unitaires avant d'écrire le code lui-même.
- 3 étapes:
 - Ecrire le test
 - Il doit échouer
 - Ecrire le code minimal
 - Le test doit réussir
 - « Refactorer » le code

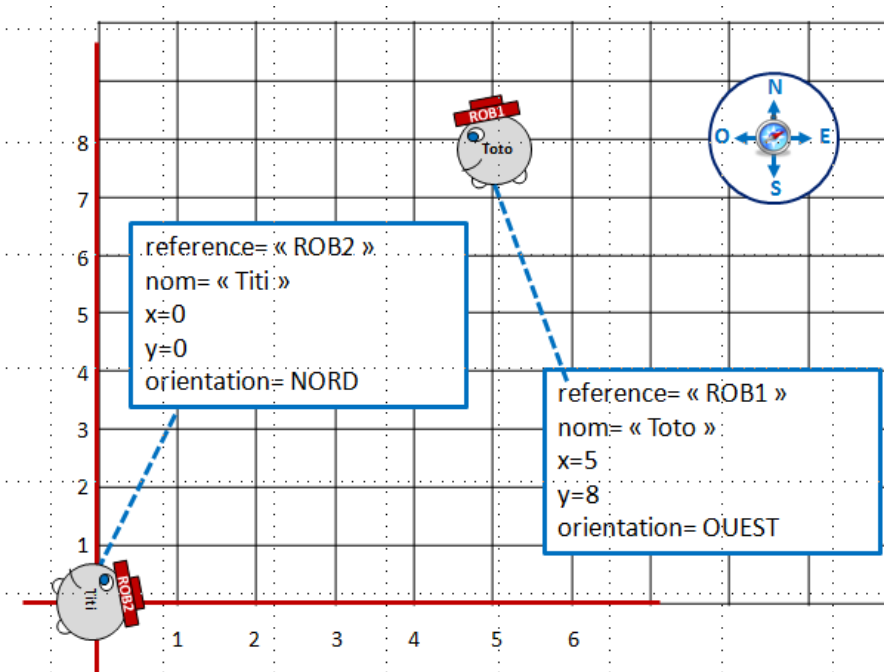


Pourquoi écrire les tests avant le code?

- Une garantie de qualité!
- Préciser la fonctionnalité à implémenter
- Servir de base à la discussion avec le client
- Constituer une base de tests de non-régression
- Faciliter la maintenance des tests
- Un investissement au départ pour un bénéfice important tout au long du développement!

Exemple - Une classe Robot

- Objectif: une classe Robot
 - Méthode setOrientation
 - Méthode déplacer



```
public class Robot
{
    private String reference;
    private int x;
    private int y;
    private String nom;
    private TypeOrientation orientation;
    public static int nbrobots=0;

    public Robot(String nom,int x, int y, TypeOrientation orientation){
    }
    public void deplacer(){
    }
    public String getReference() {
        return null;
    }
    public void setReference(String reference) {
    }
    public int getX() {
        return 0;
    }
    public int getY() {
        return 0;
    }
    public String getNom() {
        return null;
    }
    public TypeOrientation getOrientation() {
        return null;
    }
    public void setOrientation(TypeOrientation orientation) {
    }
    public String toString(){
        return null ;
    }
}
```


Etape 1 - Ecrire les Tests

```
class RobotTests {
    @Test
    final void changerOrientationTest() {
        Robot r=new Robot("Toto",10,20,TypeOrientation.SUD);
        r.setOrientation(TypeOrientation.NORD);
        assertEquals(r.getOrientation(),TypeOrientation.NORD);
    }

    @Test
    @DisplayName("Déplacement vers le SUD")
    final void deplacerTestSud() {
        Robot r=new Robot("Toto",10,20,TypeOrientation.SUD);
        r.deplacer();
        assertEquals(r.getX(),10);
        assertEquals(r.getY(),19);
    }

    @Test
    @DisplayName("Déplacement vers le NORD")
    final void deplacerTestNord() {
        Robot r=new Robot("Toto",10,20,TypeOrientation.NORD);
        r.deplacer();
        assertEquals(r.getX(),10);
        assertEquals(r.getY(),21);
    }

    @Test
    @DisplayName("Déplacement vers l'EST")
    final void deplacerTestEst() {
        Robot r=new Robot("Toto",10,20,TypeOrientation.EST);
        r.deplacer();
        assertEquals(r.getX(),11);
        assertEquals(r.getY(),20);
    }

    @Test
    @DisplayName("Déplacement vers l'OUEST")
    final void deplacerTestOuest() {
        Robot r=new Robot("Toto",10,20,TypeOrientation.OUEST);
        r.deplacer();
        assertEquals(r.getX(),9);
        assertEquals(r.getY(),20);
    }
}
```

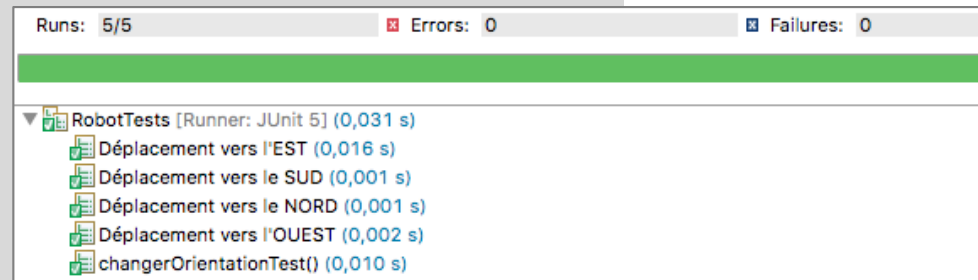
Echec des
tests

Runs:	5/5	Errors:	0	Failures:	5
RobotTests [Runner: JUnit 5] (0,047 s)					
x Déplacement vers l'EST (0,033 s)					
x Déplacement vers le SUD (0,003 s)					
x Déplacement vers le NORD (0,003 s)					
x Déplacement vers l'OUEST (0,002 s)					
x changerOrientationTest() (0,003 s)					

Etape 2 - Coder « au plus vite »

- Objectif: faire réussir les tests

```
public class Robot
{
    private String reference;
    private int x;
    private int y;
    private String nom;
    private TypeOrientation orientation;
    public static int nbrobots=0;
    public Robot(String nom,int x, int y, TypeOrientation orientation)
    {
        nbrobots++;
        this.reference="ROB" + nbrobots;
        this.x = x;
        this.y = y;
        this.nom = nom;
        this.orientation = orientation;
    }
    public void deplacer()
    {
        switch (orientation) {
            case NORD: y++; break;
            case EST: x++; break;
            case SUD: if (y>0) y--; break;
            case OUEST:if (x>0) x--; break;
        }
    }
    .....
}
```



Etape 3 - Améliorer la qualité du code

- Refactoring
- Prendre le temps de remanier le code pour augmenter sa qualité :
 - Éliminer le code dupliqué
 - Se conformer aux conventions de nommage et de présentation
 - Rendre le code plus simple et plus lisible
 - Rendre le code plus évolutif

Bonnes pratiques
Principes SOLID

Exercice : classe mystère



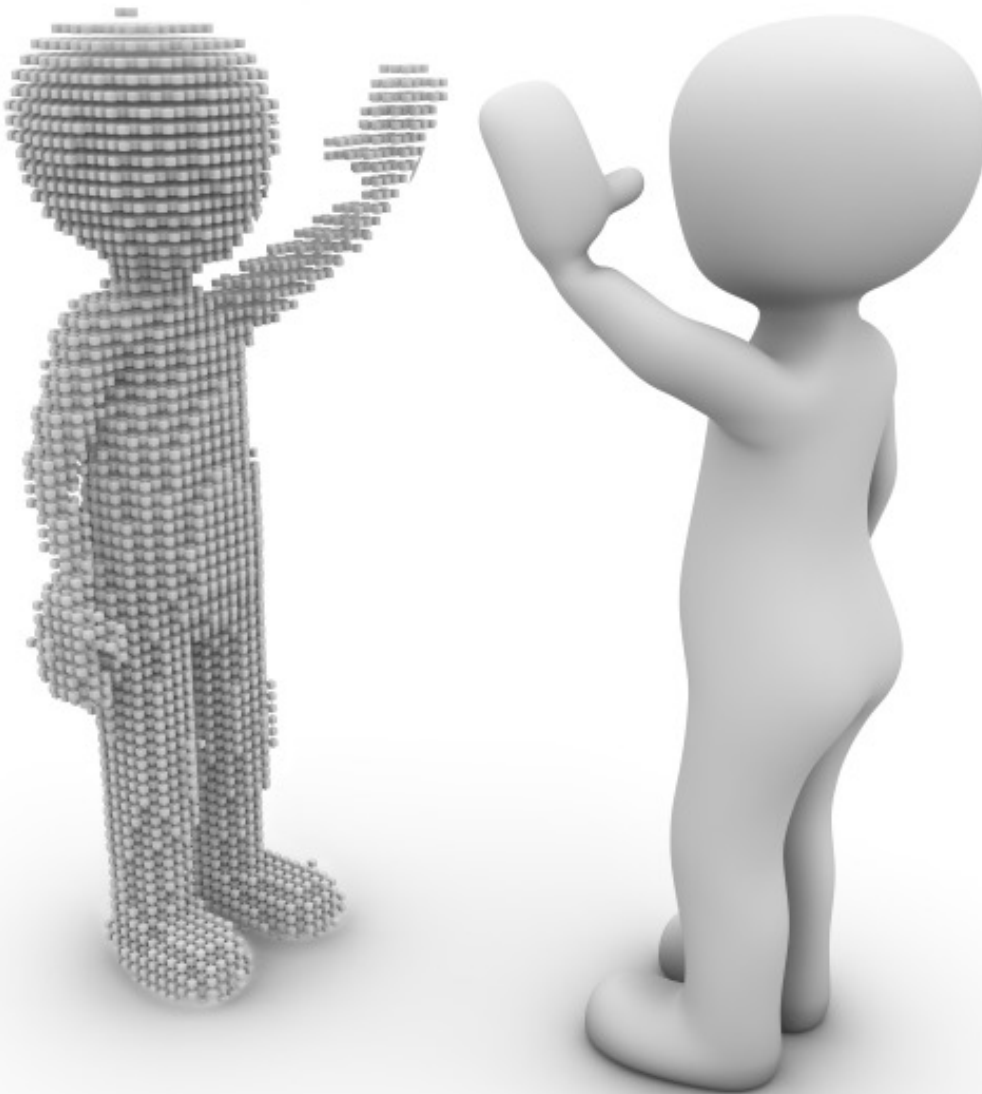
Appliquez la démarche TDD pour définir une classe Mystère conforme aux spécifications ci-dessous:

1. Définir la classe de test
2. Exécuter le test qui échoue
3. Ecrire le code pour faire réussir les tests
4. Améliorer l'écriture du code

Pour la méthode *generer*, le test doit montrer:

- que les valeurs générées sont bien comprises en 0 et valeurMax
- qu'il y a au moins une occurrence de chaque valeur entre 0 et valeurMax

```
public class Mystere {
    private int valeur; // La valeur du nb mystère
    private int valeurMax; // La valeur maximum possible
    public Mystere(int valeurMax ) { }
    public int getValeurMax() {return 0;}
    public int getValeur() {return 0;}
    public void setValeur(int valeur) {}
    public void generer() {
        //génère une valeur aléatoire comprise entre 0 et valeurMax et
        //l'affecte à la valeur du nombre mystère.
    }
    public int testProp(int proposition){
        //renvoie 1 si le nombre est trouvé, 0 s'il est plus petit et
        //2 s'il est plus grand
        return 0;}
}
```

Mocking

- Principe
- Mise en œuvre avec Mockito



Notion de mock



- Les **mocks** sont des objets factices (*doublures*) qui permettent de remplacer des instances de classes avec lesquelles la classe à tester communique:
 - Classe non encore implémentée
 - Classe non accessible
- Objectif: tester en « **isolation** »

Les mocks sont utilisés dans le cadre d'une démarche TDD mais peuvent être aussi utilisés dans des tests unitaires classiques.

Principe des mocks

- Un mock a la même interface que l'objet qu'il simule.
- Un **framework de mock** permet:
 - de créer des mocks
 - Méthode **mock** ou annotation **@mock**
 - de spécifier leur comportement (méthodes) valeurs de retour ou levée d'exception
 - Méthode **when**
 - d'interroger les mocks à la fin du test pour savoir comment ils ont été utilisés
 - Méthode **verify**

Exemples de framework en Java: EasyMock,JMockIt,Mockito,JMock,MockRunner.

Le framework Mockito

- Générateur automatique de mock en java
- Compatible avec JUnit
- Installation
 - Télécharger le fichier d'installation .jar
 - <https://jar-download.com/artifacts/org.mockito>
 - Intégrer le .jar dans les *libraries* du projet Java



Création de mocks et stubbing

- Création d'un objet « **mocké** » d'une classe A
 - `objA=Mockito.mock(A.class)`
- Le « **stubbing** » consiste à remplacer le comportement d'une méthode :
 - Méthode qui a un type de retour
 - `Mockito.when(objA.m(..)).thenReturn(valeur) ;`
 - `Mockito.when(objA.m(..)).toThrow(exception) ;`
 - Méthode de type void :
 - `Mockito.doThrow(exception).when(objA.m(..))`

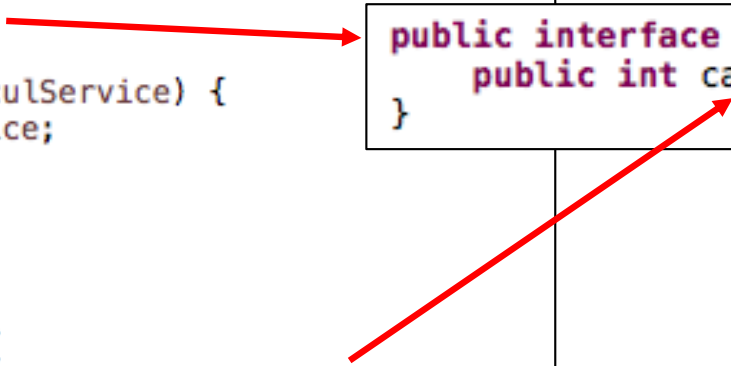
```
Point pMoc; pMoc=Mockito.mock(Point.class);  
when(pMoc.getAbcisse()).thenReturn(2);  
when(pMoc.setAge(-5)).toThrow(new AgeIllegalException());  
assertEquals(pMoc.getAbcisse()*10,20);
```

Exemple de test avec mock

- On souhaite définir une méthode addCarres() dans la classe Calculator.
- Cette méthode utilise la méthode carre() de l'interface CalculService.

```
public class Calculator {  
    private CalculService calculService;  
  
    public Calculator(CalculService calculService) {  
        this.calculService = calculService;  
    }  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public int addCarres(int a, int b) {  
        return add(calculService.carre(a), calculService.carre(b) );  
    }  
}
```

```
public interface CalculService {  
    public int carre(int x);  
}
```



Exemple de test avec mock

```
public class CalculatorTddTests {  
  
    private Calculator calculator;  
    private CalculService calculService;  
  
    @BeforeEach  
    void setUp() {  
        calculator=new Calculator(calculService);  
    }  
    @DisplayName("Test addition carre")  
    @Test  
    void testSommeCarre() {  
        assertTrue(calculator.addCarres(2,3)==13, "calcu'  
    }  
}
```

- Le test déclenche une erreur car l'objet calculService n'est pas instancié.

Runs: 1/1 Errors: 1 Failures: 0

CalculatorTddTests [Runner: JUnit 5] (0,025 s)

test somme carre (0,025 s)

Failure Trace

java.lang.NullPointerException: Cannot invoke "tddCalcul.CalculService.carre(int)" because "this.calculService" is null

at tddCalcul.Calculator.addCarres(Calculator.java:11)

at tests.CalculatorTddTests.testSommeCarre(CalculatorTddTests.java:29)

Pour corriger cette erreur on va utiliser un Mock pour remplacer l'instance attendue

Exemple de test avec mock

```
import org.mockito.Mockito;

class CalculatorTddTests {

    private Calculator calculator;
    private CalculService calculService;

    @BeforeEach
    void setUp() {
        calculService=Mockito.mock(CalculService.class);
        calculator=new Calculator(calculService);
    }

    @DisplayName("Test addition carre")
    @Test
    void testSommeCarre() {
        Mockito.when(calculService.carre(2)).thenReturn(4);
        Mockito.when(calculService.carre(3)).thenReturn(9);
        assertTrue(calculator.addCarres(2,3)==13, "calcul exact");
    }

}
```

Instanciation du mock

Indique ce que doit retourner la méthode carre pour 2 et 3

Runs: 1/1 ✖ Errors: 0 ❌ Failures: 0

CalculatorTddTests [Runner: JUnit 5] (0,144 s)
 Test addition carre (0,144 s)



Exercice : mock Mystere

En supposant que la classe Mystere n'a pas encore été programmée, définissez une classe de test de la classe Joueur (méthode jouer) en utilisant un mock pour l'objet Mystere.

```
public class Joueur {
    private String nom;
    private Mystere nombreMystere;

    public Joueur(String nom) {
        this.nom=nom;
    }
    public void initialiserJeu(int valMax) {
        this.nombreMystere=new Mystere(valeurMax);
    }
    public void setNombreMystere(Mystere m) {
        nombreMystere =m;}

    public String jouer(int prop) {
        int res=this.nombreMystere.testProp(prop);
        String msg="PlusPetit";
        if (res==1) {
            msg="gagne";
        }
        else if (res==2) {
            msg="PlusGrand";
        }
        return msg;
    }
}
```



Exercice : mock Mystere

```
public class Mystere {  
    private int valeur; // La valeur du nb mystère  
    private int valeurMax; // La valeur maximum possible  
    public Mystere(int valeurMax ) { }  
    public int getValeurMax() {return 0;}  
    public int getValeur() {return 0;}  
    public void setValeur(int valeur) {}  
    public void generer() {  
        //génère une valeur aléatoire comprise entre 0 et valeurMax et  
        //l'affecte à la valeur du nombre mystère.  
    }  
    public int testProp(int proposition){  
        //renvoie 1 si le nombre est trouvé, 0 s'il est plus petit et  
        //2 s'il est plus grand  
        return 0;}  
}
```



Exercice : Produit et Inventaire

1. Récupérez sur l'ENT les classes Produit, Inventaire et TestInventaire.
2. Exécutez la classe TestInventaire (classe de test Junit) afin de vérifier qu'un des tests échoue.
3. Définissez une deuxième version de la classe TestInventaire (TestInventaireMock) qui n'utilise pas des instances de produit mais des mocks de produit.
4. Exécutez votre classe TestInventaireMock et vérifiez que les tests réussissent.

Cela permet de mettre en évidence
que l'erreur n'est pas dans Inventaire !!



Exercice : Produit et Inventaire

5. Définissez une classe de test unitaire Junit de la classe Produit afin de mettre en évidence les erreurs en vérifiant :
 - que des exceptions sont levées en cas de valeurs négatives des quantités et id dans le constructeur et dans les méthodes set
 - que le setQuantite fonctionne correctement.
6. Corrigez la classe Produit et vérifiez qu'à présent les tests réussissent :
 - tests unitaires de Produit
 - test initial InventaireTest